

From TE-CCL to Graph Neural Policy Learning for Adaptive Collective Communication

Problem: Optimizing collective communication over heterogeneous accelerator topologies — especially in multi-chassis or cross-region-like settings— is computationally expensive.

Current solution: TE-CCL uses MILP optimization to generate high-quality schedules, but solving can be computationally expensive.

Our goal: Learn this optimization behavior with graph neural networks in a using a "Teacher-Student" framework , so that high-quality communication decisions can be produced much faster



Pipeline Overview

Step 1 — Teacher data extraction

TE-CCL outputs are parsed into:

- topology graphs
- policy matrices
- collective finish times

Step 2 — Agent

Input: topology graph

Output: edge-level communication policy

Step 3 — SimNet

Input: topology graph + policy

Output: predicted collective completion time

End-to-end goal

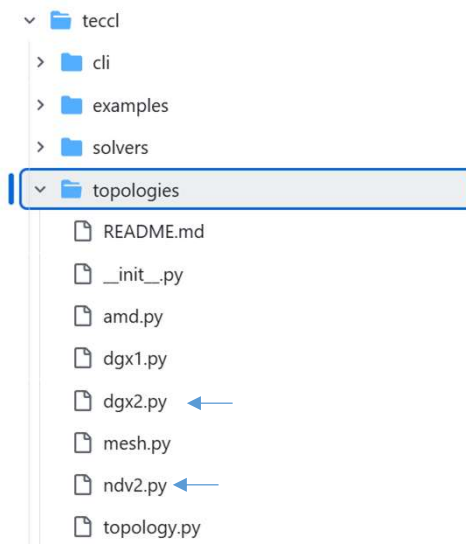
Topology → Agent → Policy → SimNet → Time



TE-CCL Repository as Teacher and Data Source

2 components of the TE-CCL repository were used:

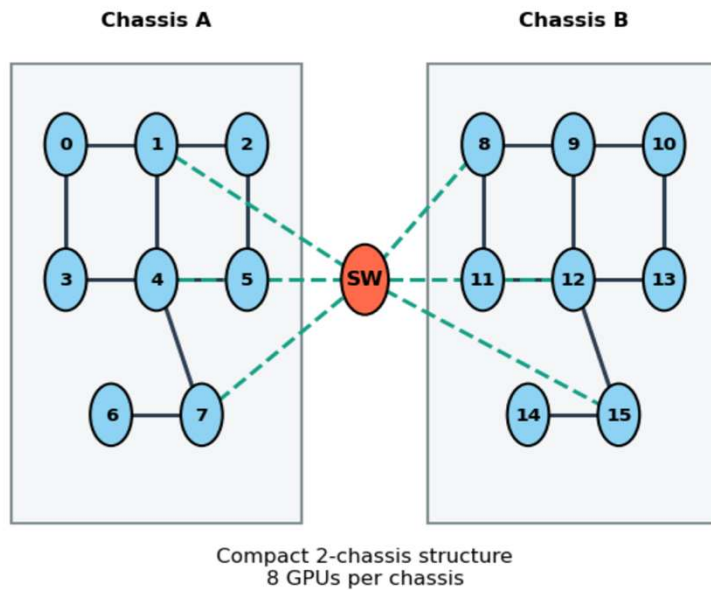
1. **Topologies** : To construct communication graphs



- NDv2 and DGX2 were the two topology families used in the final dataset, since they are experiments with them provided with the TE-CCL expert outputs.
- Topologies were defined with :
 - Capacity** → link bandwidth
 - Alpha** → link latency
 - Switch indices** → switch node identities
 - Chassis** → multi-chassis structure
 - Nodes per chassis** → local topology size

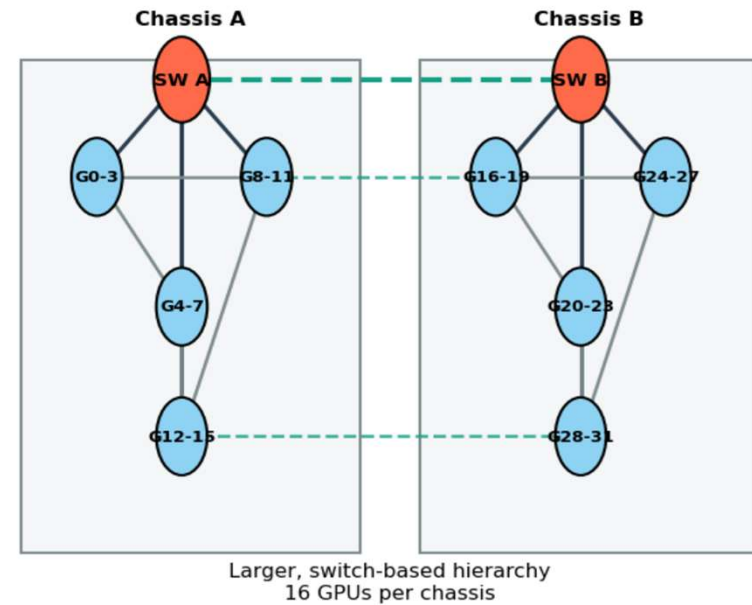
Topologies Used in Final Dataset

NDv2 Topology Abstraction



Schematic view: node positions are illustrative; dashed links indicate cross-chassis communication.

DGX2 Topology Abstraction



Schematic abstraction: GPU groups are aggregated for readability; dashed links indicate cross-chassis interconnects.



TE-CCL Repository as Teacher and Data Source

2. Experiments with Provided outputs : To extract expert schedules and collective completion times as supervision targets

Each output file represents a combination of configurations:



Component	Final Dataset
Topologies	NDv2, DGX2
Chassis	2, 4 (4-chassis mainly from NDv2)
Collectives	AllGather, AlltoAll
Modes	Fast, Slow, Fast_Early_Stop
Message sizes	1KB – 1GB
Total samples	154



Graph Construction from Topology

Each topology is converted into a directed graph :

An ***edge_{ij}*** $\in E$ exists if,

$$\{\mathbf{capacity}\}_{\{ij\}} > 0$$

Each edge is represented by:

$$\mathbf{edge}_{ij} = [\mathbf{capacity}_{ij}, \alpha_{ij}, \mathbf{switch}_{ij}]$$

where,

$$\alpha_{ij}: \text{link latency} \quad \mathbf{switch}_{ij} \in \{0, 1\}: \text{switch involvement}$$

So, the graph is fully defined by:

- nodes (GPUs and switches)
- edges (communication links)
- edge attributes (bandwidth, latency, switch indicator)



Policy Matrix Extraction from TE-CCL Flows

Each TE-CCL output provides a set of communication flows of the form:

Chunk c traveled over $i \rightarrow j$

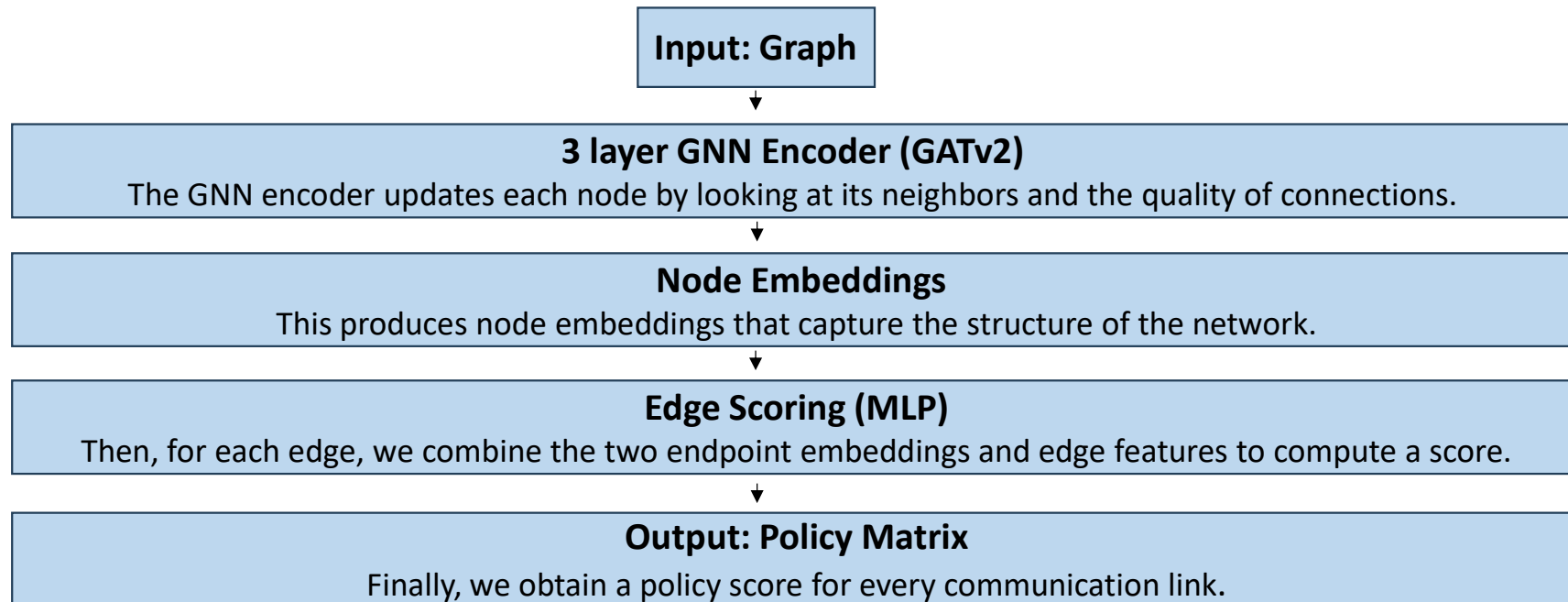
The policy matrix $\mathbf{P} \in \mathbf{R}^{N \times N}$ constructed as :

$$P_{ij} = \sum_{\text{flows}} \mathbf{1}_{\{\text{edge } i \rightarrow j \text{ is used}\}}$$

Then normalized,

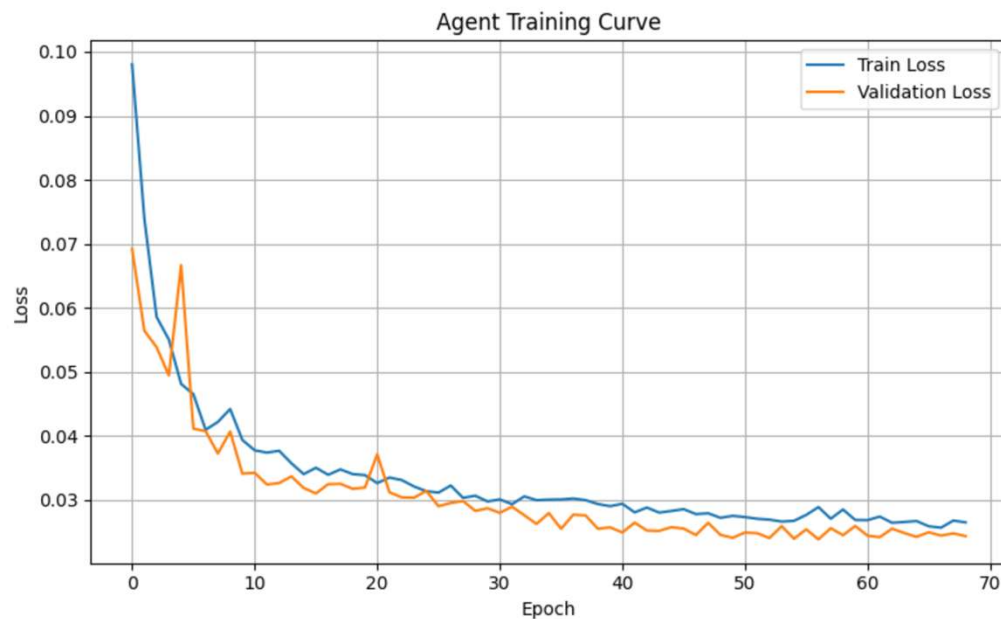
$$\widehat{P}_{ij} = \frac{P_{ij}}{\max_{a,b} P_{ab}} \quad \widehat{P}_{ij} \in [0, 1]$$

The Agent GNN



GATv2 is preferred as an attention mechanism and it allows the model to weight neighbors differently based on the varying edge attributes.

Agent Training Results



Best validation loss: 0.0237

Test loss: 0.0228

- Early stopping applied.
- The training and validation losses decrease consistently, indicating stable learning.
- The gap between them remains small, suggesting good generalization.
- The model alone successfully learns to approximate the expert communication policies.



SimNet GNN: The Proxy Model

- **Goal:** Learn a **differentiable surrogate model** of collective communication time and approximate expensive simulator calls during optimization.

Input:

- Graph topology (nodes + edges)
- Edge features (latency, bandwidth, policy)
- Global features:
 - Message size (log-scale)
 - Collective type (AllGather / AlltoAll)

Output:

- Predicted completion time
(via residual reconstruction)

Key Idea: Residual Learning

Direct prediction of time is highly unstable due to scale differences. Instead problem is decomposed the into a baseline and a residual.

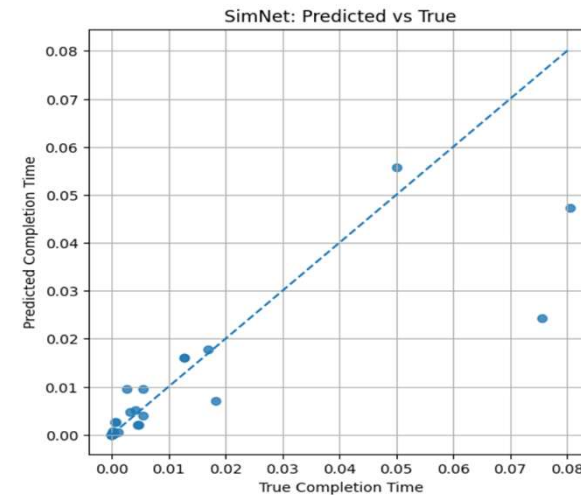
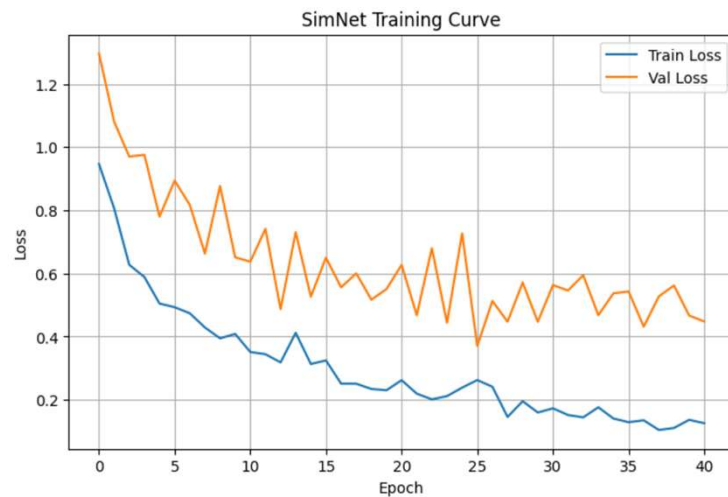
$$\text{Completion time} = \text{baseline} \times \text{residual}$$

(baseline $\approx$ message size / link capacity)

- Baseline captures coarse scaling behavior
- Model focuses on **topology-specific deviations**
- **Model predicts log(residual) for stability**



SimNet Training Results



- Model achieves nearly stable convergence during training, **validation loss fluctuates**.
- Good alignment between predicted and true completion times at small scale.
- *Higher variance observed for larger message sizes*, likely due to increased complexity and **limited training samples in that regime**.
- *Performance degrades slightly in high-latency / large-scale regimes*
- *Limited training data restricts full generalization*